

Local LLM · RAG · Agent

Trợ lý AI chạy 100% trên máy của bạn

2 giờ · 3 module · Hands-on · Không byte nào rời máy

Vì sao chạy LLM trên máy mình?

Riêng tư

Dữ liệu **không** rời máy — hợp đồng, mã nguồn, hồ sơ, bài tập

Độc lập

Pull về xong → **offline vẫn chạy**, không phụ thuộc mạng/nhà cung cấp

Miễn phí

Không token, không giới hạn, không hoá đơn theo lượt gọi

Trực tuyến suốt buổi học: mỗi lần bạn dán nội dung vào ChatGPT, nó rời khỏi máy bạn lên server công ty khác. Local LLM cho bạn đúng năng lực đó — **không gửi đi đâu cả**.

Bạn thấy mình trong đó chứ?

- **Văn phòng** → hợp đồng, bảng lương, danh sách khách hàng
- **Lập trình viên** → mã nguồn nội bộ, API key
- **Nhà nghiên cứu** → dữ liệu, bản thảo chưa công bố
- **Sinh viên** → đề án, bài tập của mình
- **Y tế / pháp lý** → hồ sơ bệnh án, hồ sơ thân chủ
- **Tổ chức** → tài liệu nội bộ nhạy cảm

Bằng chứng vật lý (demo giữa buổi): tắt Wi-Fi → AI vẫn trả lời. Không mạng, không cloud, không gửi đi đâu.

Roadmap — 3 module, trọng số rõ ràng

Phần	Vai trò	Thời lượng
1. Local LLM với Ollama	★ Trục chính	44' (giảng + thực hành kỹ)
2. RAG cho tài liệu của bạn	Thứ yếu, quan trọng	34' (lý thuyết gọn + thực hành)
3. Agent	Giới thiệu — về tự khám phá	18' (chỉ demo)

Local LLM (chạy được) → RAG (cho LLM đọc tài liệu của bạn) → Agent (cho LLM biết hành động)

PHẦN 1 / 3 · TRỰC CHÍNH

Local LLM với Ollama

Ollama = "Docker cho LLM"

Docker

```
docker pull nginx  
docker run nginx
```

Ollama

```
ollama pull qwen3:1.7b  
ollama run qwen3:1.7b
```

Ẩn hết phần khó:

- ✗ Không cần biết CUDA
- ✗ Không cần config VRAM
- ✗ Không cần quản batching
- ✗ Không cần compile

Chính thức:

ollama.com · 90K+ ★

5 ưu điểm nhớ ngay

1 Cài 1 lệnh

```
winget install Ollama.Ollama
```

2 Tự tối ưu phần cứng

Auto detect GPU/CPU, không cần config

3 Đổi model 1 dòng

```
ollama.chat(model="qwen3:1.7b")  
# → ollama.chat(model="qwen3:4b")
```

4 API tương thích OpenAI

```
client = OpenAI(  
    base_url="http://localhost:11434/v1",  
    api_key="ollama"  
)
```

→ Không khoá cứng vào 1 nhà cung cấp

5 Offline 100%

Pull về xong → tắt mạng vẫn chạy

Demo: 5 dòng code = LLM chạy local

```
import ollama

response = ollama.chat(
    model="qwen3:1.7b",
    messages=[
        {"role": "system", "content": "Bạn là trợ lý hữu ích, trả lời ngắn gọn."},
        {"role": "user", "content": "Giải thích RAG trong 3 câu."},
    ],
    think=False,  # tắt thinking mode của Qwen3 → output sạch
)
print(response.message.content)
```

Chạy là có ngay câu trả lời — không cần CUDA, không cần quản VRAM. Xem thêm ở demo RAG (slide sau) + notebook Module 1.

Chọn model theo MÁY của bạn

RAM máy	Model gợi ý	Size	Tốc độ CPU
8 GB	qwen3:1.7b 	1.4 GB	73 tok/s
16 GB	qwen3:4b	2.5 GB	~77 tok/s
16 GB+	llama3.2:3b	2.0 GB	~60 tok/s
GPU / 32 GB	qwen3:8b	5.2 GB	~40 tok/s

Quantization (Q4): nén model xuống 4-bit → nhẹ 4× mà giữ ~97% chất lượng. Đó là lý do model 7 tỷ tham số chạy được trên laptop 8 GB. **Đổi model = đổi 1 string.**

Bản đồ thuật ngữ (cho người mới)

Thuật ngữ	Hiểu nôm na
Model	"Bộ não" AI đã huấn luyện sẵn (file vài GB)
Token	Mẫu chữ (~¾ từ). LLM sinh từng token một
Tham số (1.7B, 4B...)	Số "nơ-ron" — càng nhiều càng thông minh & nặng
Embedding	Biến chữ thành dãy số để máy so sánh nghĩa
Temperature	0 = chắc chắn, nhất quán · 1 = sáng tạo, ngẫu nhiên
Context window	"Trí nhớ ngắn hạn" — số token model đọc được 1 lần

Không cần thuộc lòng — tra lại bảng này khi gặp trong code.



Thực hành Phần 1 · 22 phút

Thành quả: "LLM chạy offline trên máy TÔI"

1. `ollama --version` + `ollama list` — kiểm tra cài
2. `ollama run qwen3:1.7b` → hỏi 1 câu chủ đề của bạn
3. Notebook Python 5 dòng → đổi system prompt thành persona riêng
4. Sửa **Modelfile** → build model "của bạn" (tính cách + trả lời ngắn)
5. (Thêm) chỉnh `temperature` thấp ↔ cao, quan sát khác biệt

PHẦN 2 / 3 · THỨ YẾU NHƯNG QUAN TRỌNG

RAG — cho LLM đọc tài liệu của bạn

Bài toán: AI không biết tài liệu của BẠN

Bạn có một đồng tài liệu của riêng mình — sổ tay, quy trình, ghi chú, giáo trình.
ChatGPT không biết gì về chúng. Và bạn cũng không muốn upload hết lên cloud.

❌ LLM thuần

"Hỏi một người thông minh **nhưng không biết** tài liệu của bạn"
→ trả lời chung chung / bịa

✅ LLM + RAG

"Cho người đó **tra tài liệu của bạn trước** khi trả lời"
→ bám sát tài liệu thật, **trích nguồn**

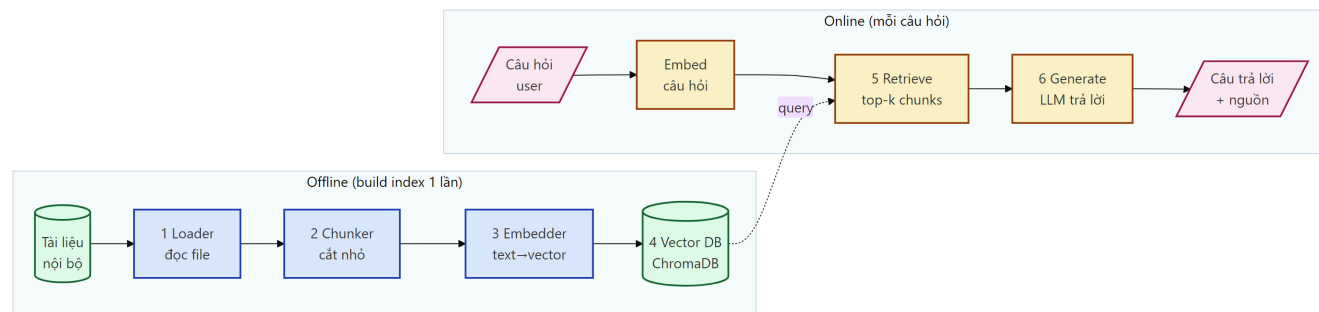
RAG pipeline — 5 bước

Offline (build 1 lần):

1. **Load** — đọc tài liệu
2. **Chunk** — cắt ~500 ký tự
3. **Embed** — text → vector
4. **Store** — lưu vector DB

Online (mỗi câu hỏi):

5. **Retrieve** — tìm top-k đoạn
6. **Generate** — LLM trả lời + nguồn



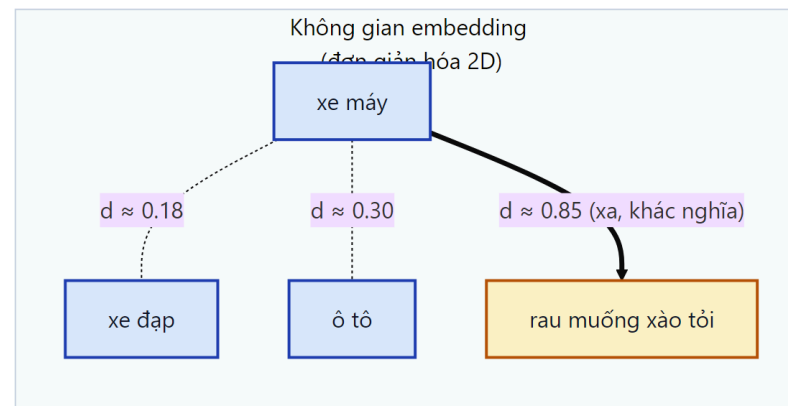
Embedding — bước "ma thuật"

Model ánh xạ **text** → **vector** ~768 chiều

Quy tắc:

text có nghĩa gần nhau
→ vector gần nhau

Hỏi "xe đạp" tìm được "xe máy" (gần nghĩa), không tìm "rau muống". **Keyword search** sẽ trượt — **embedding** thì hiểu nghĩa.



Text có nghĩa gần nhau
→ vector gần nhau
→ distance nhỏ

Demo RAG

```
PowerShell - RAG

$ python 2_rag/rag_minimal.py --ask "Quy trình xử lý sự cố ATTT gồm những bước nào?"

[Câu hỏi] Quy trình xử lý sự cố ATTT gồm những bước nào?
[Retrieve] Đang tìm top-3 đoạn liên quan...

[Sinh câu trả lời]

Quy trình xử lý sự cố ATTT được quy định trong Quy trình
6 bước (NIST SP 800-61) như sau:

1. Bước 1 - Chuẩn bị (Preparation)
- Duy trì danh sách liên hệ ứng cứu (CIRT)
- Cập nhật playbook cho từng loại sự cố
- Diễn tập 6 tháng/lần

2. Bước 2 - Phát hiện và phân tích (Detection & Analysis)
- Tiếp nhận cảnh báo từ SIEM, IDS, người dùng báo cáo
- Xác định mức độ ưu tiên (P1-P4)
- Thu thập bằng chứng ban đầu (log, memory dump, snapshot)

3. Bước 3 - Cách ly (Containment)
- Cách ly ngắn hạn: chặn IP, disable account...

Trích nguồn: 03_quy_trinh_su_co.md

[STATUS] Model: qwen3:1.7b · Embedding: nomic-embed-text · ~18s · 100% offline
```

Kết quả trên terminal

🔒 **Trợ lý tài liệu nội bộ**

Hệ thống RAG chạy 100% local - LLM: qwen3:1.7b · Embedding: nomic-embed-text

🤖 **CẤU HÌNH LLM**

Model

Đổi model live, không cần restart

qwen3:1.7b

Top-k đoạn tham khảo

3

Cao → context phong phú nhưng tốn token

110

☒ Hiện đoạn tham khảo dưới câu trả lời

📁 **VECTOR INDEX**

Trạng thái

☒ Index sẵn sàng — 25 chunks

📁 **TÀI LIỆU TRONG KHO**

01_quy_dinh_mat_khau.md

02_phan_loai_tai_lieu.md

Hỏi về quy định nội bộ...

Gửi

Examples

	Model	Top-k đoạn tham khảo	Hiện đoạn tham khảo dưới câu trả lời
Quy trình xử lý sự cố ATTT gồm những bước nào?	qwen3:1.7b	3	true
USB cá nhân có được dùng không?	qwen3:1.7b	3	true
Quy định mật khẩu của đơn vị?	qwen3:1.7b	3	true
Có được forward email công vụ sang gmail?	qwen3:1.7b	3	true

Giao diện Gradio

~80 dòng Gradio = chat UI streaming + chọn model + trích nguồn từng câu trả lời

© 2026 · 100% offline

16 / 23

Thực hành Phần 2 · 19 phút

Thành quả: "AI đọc tài liệu CỦA TÔI"

1. Chạy notebook RAG trên dataset mẫu → thấy trích nguồn
2. ★ Thả 1-2 file của riêng bạn (`.md` / `.txt`) vào `data/`, build lại, hỏi → AI trả lời theo tài liệu của chính bạn
3. Đổi `CHUNK_SIZE` 200 / 500 / 1000 → so kết quả (hiểu chunking)
4. (Bảo mật nhẹ) thả file chứa câu "lừa" → xem RAG có bị dẫn dắt không

PHẦN 3 / 3 · GIỚI THIỆU

Agent — cho LLM biết hành động

(Demo hôm nay — thực hành về tự khám phá)

Agent = LLM + Tools + ReAct loop

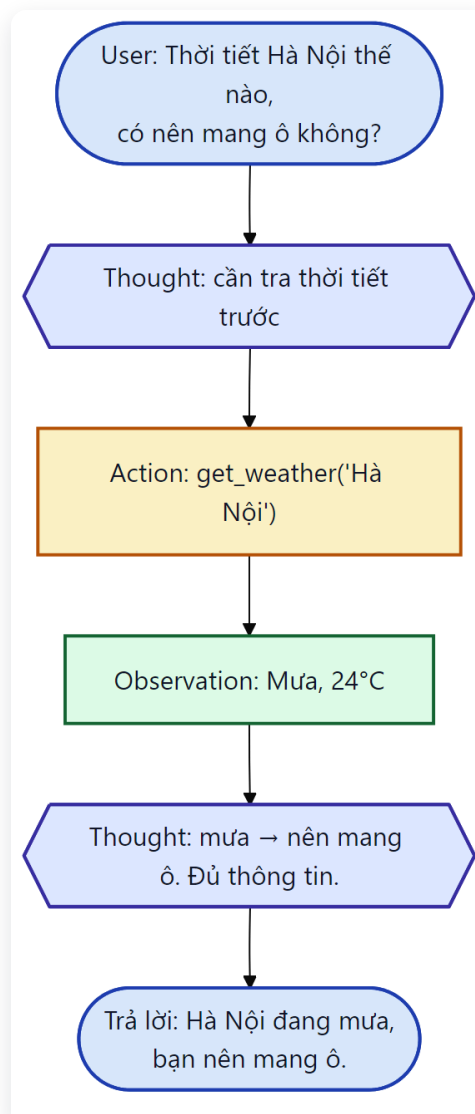
Giới hạn RAG: chỉ tra → trả lời, không hành động

Agent tự quyết gọi tool nào, dừng khi nào:

1. **Thought** — cần gì tiếp?
2. **Action** — gọi `tool(args)`
3. **Observation** — kết quả → lặp

Tool = function Python bất kỳ.

Paper gốc: **ReAct** (Princeton + Google, 2022)



Demo Agent

PowerShell - Agent ReAct

```
$ python 3_agent/agent_simple.py --ask "Bây giờ là mấy giờ?"

=====
[User] Bây giờ là mấy giờ?
=====

[Các bước agent đã thực hiện]

1. [ModelRequest] SystemPromptPart + UserPromptPart
2. [ModelResponse] ThinkingPart: 'user asking time in Vietnamese,
                    need get_current_time'
                    ToolCallPart: get_current_time()
3. [ModelRequest] ToolReturnPart:
                    '2026-05-29 09:40:15, Thứ Sáu'
4. [ModelResponse] TextPart: <final answer>

[Câu trả lời cuối]

Thời gian hiện tại là **2026-05-29 09:40:15, Thứ Sáu**.
(Kết quả được trả về từ công cụ `get_current_time`.)

[NOTE] Model 1.7B handle 1-tool tốt. Cho chain nhiều tool dùng qwen3:4b+.
[NOTE] Để xem chain multi-tool: chạy --demo (3 query có IP+RAG+log).
```

 **Hôm nay:** chỉ cần **thấy** agent hoạt động & hiểu nó khác RAG ở chỗ **biết hành động**

© 2026 · 100% offline

 **Về nhà:** thêm tool của bạn, thử multi-tool với model 4B+, đọc [log_reader.py](#) để hiểu **tool** tư bảo vệ

Sau 2 giờ bạn đã làm được

- ✓ LLM chạy 100% offline trên máy của bạn
- ✓ Chatbot đọc tài liệu của bạn, có trích nguồn
- ✓ Thấy agent hành động — và có repo để tự nghịch tiếp

3 việc làm tuần này

1. Làm RAG cho **tài liệu thật** của bạn
2. Thử **model khác** xem hợp máy mình không
3. Đọc handbook phần bạn quan tâm (RAG nâng cao / Agent / bảo mật)

Một điểm bảo mật để mang về

Khi cho AI quyền đọc file / gọi công cụ → AI có thể bị "lừa".

Vì vậy công cụ phải **tự giới hạn mình** (sandbox), đừng tin LLM mù quáng.

Đúng cho mọi app AI — không riêng lĩnh vực nào.

Đọc thêm trong handbook (đã cắt khỏi giờ giảng để gọn):

RAG poisoning · PII trong embedding · bảo mật agent · roadmap production (vLLM, Qdrant, MCP) · khi nào KHÔNG nên dùng local.

Cảm ơn các bạn

Q&A — thảo luận

 Repo: code + handbook + dataset mẫu + slide này

 Trục cốt lõi: **dữ liệu của bạn không rời máy**

 Fork repo → thay tài liệu của bạn → build trợ lý của riêng mình